

Implementing LDU-decomposition and research by data

1. Objectives

To implement LDU-decomposition and record statistics.

2. Methods

2.1 Overview of implementation

`ldu_decomposition` function, implementation of LDU-decomposition with full pivoting, contains the loop that executes the following steps in each i -th iteration:

1. Reorder matrices $\mathbf{L}$, $\mathbf{U}$ to select absolute maximal pivot element in submatrix `U[i:, i:]` [1]
2. Extract current pivot element to be contained by matrix $\mathbf{D}$ [2]
3. Compute the i -th column of unitriangular matrix $\mathbf{L}$ [3]
4. Eliminate elements of the i -th column under the current pivot element and update the submatrix `U[i + 1:, i + 1:]` [4]
5. Normalize the i -th row of matrix $\mathbf{U}$ [5]

The diagonal matrix is stored as a 1D-array and computed during loop execution; within the loop, matrices $\mathbf{L}$, $\mathbf{D}$, $\mathbf{U}$ are computed using standard operators. Time complexity of `ldu_decomposition` function adheres to LDU-decomposition ($O(n^3)$), since the function contains one main loop and nested loops for steps 1, 3, 4, 5. Two functions perform LDU-decomposition:

[2.1.1] `ldu_decomposition` function only executes LDU-decomposition of given matrix

[2.1.2] `ldu_decomposition_plus` function executes LDU-decomposition of given matrix, records statistics at each step and writes data into the specified `.bin` file.

`solve` function, implementation of solving linear equations using the obtained factors and a given column vector $\mathbf{b}$, computes the vectors $\tilde{\mathbf{b}}$, $\mathbf{Y}$, $\mathbf{Z}$, $\mathbf{W}$, $\mathbf{x}$. In computation of vectors $\tilde{\mathbf{b}}$ and $\mathbf{x}$ the features of matrices $\mathbf{P}$, $\mathbf{Q}$ are utilized [6][7]; in computation of matrices $\mathbf{Y}$ and $\mathbf{W}$ the manual implementation of dot product is used, the Kahan algorithm, FMA within it are utilized [8]. Time complexity of `solve` function adheres to theoretical framework ($O(n^2)$), since no more than two nested loops are used.

2.2 Metrics

I hereafter refer to:

[2.2.1] accuracy of computation as the Frobenius norm of the vector resulting from subtracting the computed result from the expected result:

$$\sqrt{\sum_i^m |a_i - b_i|^2}$$

[2.2.2] time of computation as difference between values of performance counter, i.e. the output of the following code:

```
import time
start = time.perf_counter()
result = func(x)
end = time.perf_counter() - start
print(end)
```

[2.2.3] contextualized accuracy of computation as the Frobenius norm of the vector obtained by dividing the difference between the computed result and the expected result by the expected result (to avoid division by zero, a small constant magnitude ε is added):

$$\sqrt{\sum_i^m \left| \frac{a_i - b_i}{b_i + \varepsilon} \right|^2}$$

[2.2.4] compensation as the following measure r of the computation error, i.e., compensation in generic compensation algorithm:

```
y = f(u, v), u = g(y, v), f(g(y, v), v) = y, g(f(u, v), v) = u #for
example, y = u + v, u = y - v; y = u * v, u = y/v, v != 0
c = f(a, b)
r = max(g(c, b) - a, a - g(c, b)) #for example, c = a/b, r = a - b * c
```

[2.2.5] compensation share as the ratio of the sum of compensations for the key computations in the i -th iteration (the steps 3, 4, 5 above) to the total sum of compensation for a single run of the algorithm:

$$\frac{r_i}{\sum_{k=1}^n r_k}$$

3. Results

3.1 Comparison with other tools

To compare the implementation (`ldu_decomposition` , `solve`) with the selected counterparts (`scipy.linalg.solve` , `numpy.linalg.solve`) the datasets of indicators on some sides of tools are provided. The datasets are produced for different types of matrices.

Metadata to the dataset:

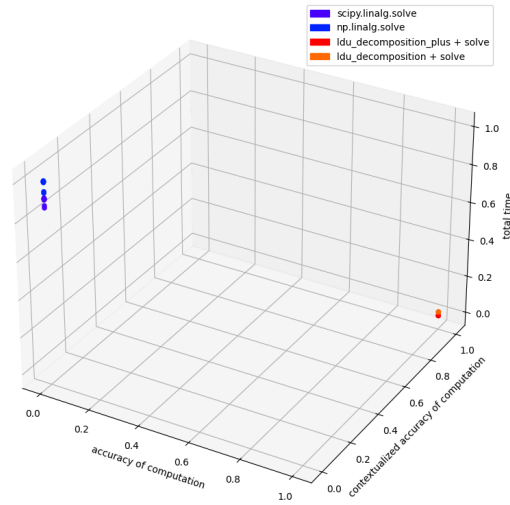
Sample size: 10 matrices of dimension 50×50 , 10 matrices of dimension 100×100 , 10 matrices of dimension 500×500 , 10 matrices of dimension 1000×1000 , 10 matrices of dimension 2000×2000

Name	Type	Definition
<code>shape</code>	<code>int16</code>	Order of given square matrix
<code>decomposing function name</code>	25-character string	Name of function factorising a given matrix
<code>solving function name</code>	20-character string	Name of function solving a given system of linear equations
<code>accuracy of computation</code>	<code>float64</code>	Accuracy of <code>solving function name</code>
<code>contextualized accuracy of computation</code>	<code>float64</code>	Contextualized accuracy of <code>solving function name</code>
<code>decomposing time</code>	<code>float64</code>	Time spent by <code>decomposing function name</code>
<code>solving time</code>	<code>float64</code>	Time spent by <code>solving function name</code>
<code>total time</code>	<code>float64</code>	Sum of <code>decomposing time</code> and <code>solving time</code>

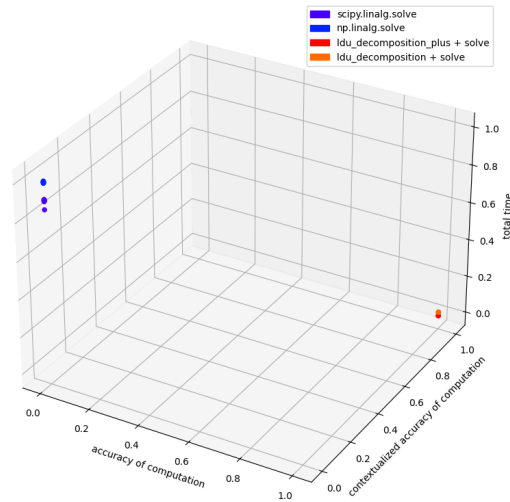
Note: If the decomposing function name and the solving function name are the same, explicit factorization is not performed (e.g., `scipy.linalg.solve` requires the coefficient matrix and yields computed solution)

Aggregation and representation of data

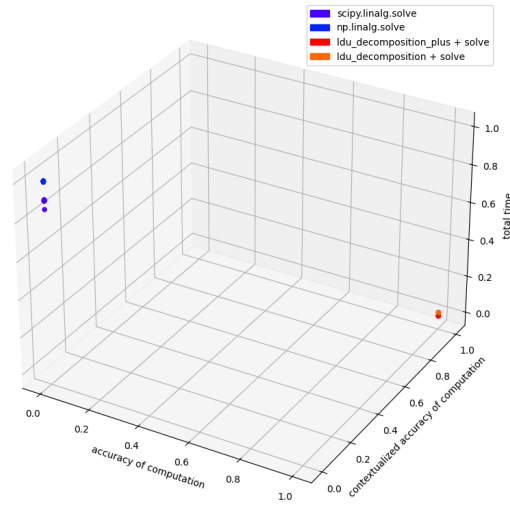
The datasets are represented below by 3D scatter plots. The $-\ln x$ function and then min-max normalization are applied to used data (`accuracy of computation` , `contextualized accuracy of computation` , `total time`); these columns determine the positions of points on x-, y- and z-axes respectively^[9]



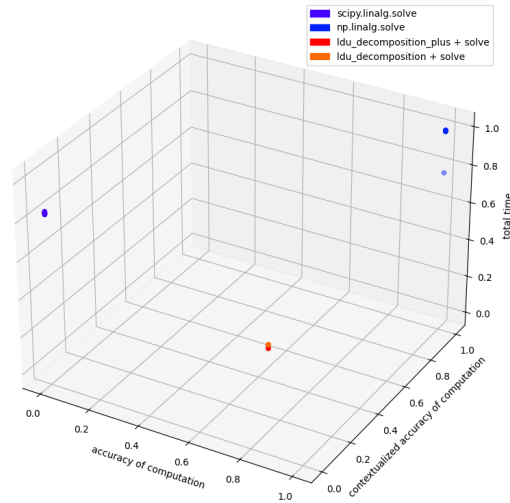
1000×1000 matrices of floating-point values from interval $[0, 1)$ (matrices returned by `numpy.random.rand(1000, 1000)`) and corresponding vectors of floating-point values from interval $[0, 1)$ (vectors returned by `numpy.random.rand(1000)`) ^[10]



1000×1000 matrices of floating-point values from interval $(-100, 100)$ (matrices returned by `numpy.random.rand(1000, 1000) + numpy.random.randint(-99, 99, size=(1000, 1000))`) and corresponding vectors of floating-point values from interval $(-100, 100)$ (vectors returned by `numpy.random.rand(1000) + numpy.random.randint(-99, 99, size=1000)`) ^[11]



1000×1000 matrices of integer values from interval $[-100, 100]$ (matrices returned by `numpy.random.randint(-100, 100, size=(1000, 1000)).astype(np.float64)`) and corresponding vectors of integer values from interval $[-100, 100]$ (vectors returned by `numpy.random.randint(-100, 100, size=1000).astype(np.float64)`)^[12]



1000×1000 Hilbert matrices (matrices returned by `scipy.linalg.hilbert(1000)`) and corresponding vectors of floating-point values from interval $[0, 1]$ (vectors returned by `numpy.random.rand(1000)`)^[13]

3.2 Statistics

To reveal statistics on various aspects of solving linear equations using the implementation (`ldu_decomposition_plus`, `solve`) the datasets of indicators for multiple solves are provided. The datasets are produced for different types of matrices. Metadata to the dataset:

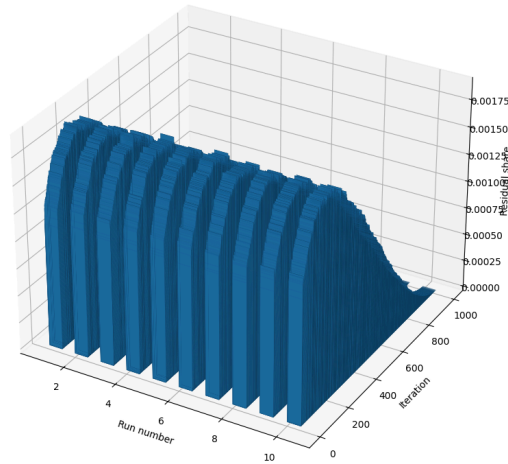
Sample size: 10 matrices of dimension 50×50 , 10 matrices of dimension 100×100 , 10 matrices of dimension 500×500 , 10 matrices of dimension 1000×1000 , 10 matrices of dimension 2000×2000

Name	Type	Definition
ts	32-character string	A combination of: 1. a time expressed in seconds since the epoch and converted to local time in the format "%d.%m.%Y %H:%M:%S:%f", i.e., the output of <code>datetime.strftime(datetime.now(), "%d.%m.%Y %H:%M:%S:%f")</code> 2. a 4-digit random number, i.e., the output of <code>datetime.strftime(datetime.now(), "%d.%m.%Y %H:%M:%S:%f") + ":" + f"{random.randint(0, 9999):04d}"</code>
salg	int16	iteration number
miarowind	int16	difference between row index of absolute maximal pivot element and row index of current pivot element
miacolind	int16	difference between column index of absolute maximal pivot element and column index of current pivot element
pivot_ratio	float64	absolute ratio of absolute maximal pivot element to current pivot element
miaresccl	float64	absolute maximal compensation in computation of unitriangular matrix L
amresccl	float64	absolute mean compensation in computation of unitriangular matrix L
miaresup	float64	absolute maximal compensation in updating of the submatrix $U[i + 1:, i + 1:]$
amresup	float64	absolute mean compensation in updating of the submatrix $U[i + 1:, i + 1:]$
miaresn	float64	absolute maximal compensation in normalization of the i -th row in matrix U
amresn	float64	absolute mean compensation in normalization of the i -th row in matrix U

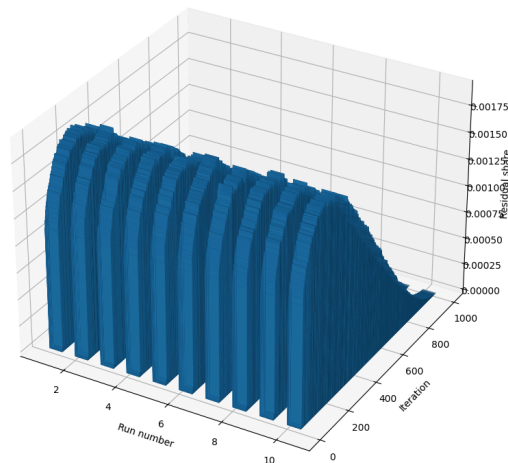
Note: The actual length of `ts` is 31 characters, but 32nd character is intended for a null character

Application. Distribution of compensations across iterations

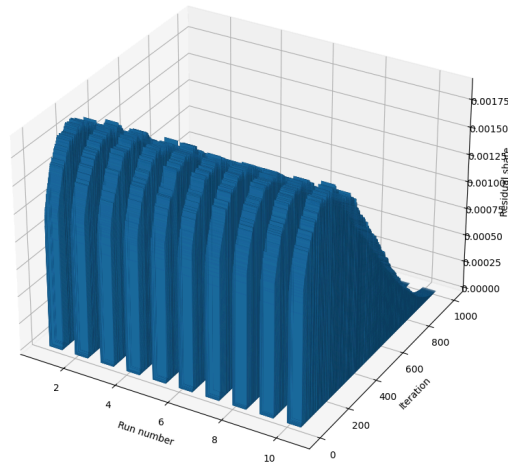
From the absolute mean compensations `amresccl` , `amresup` , `amresn` in the steps of the i -th iteration, compensation share r_i of each i -th iteration can be obtained.^[14]
The results are represented below by 3D histograms.



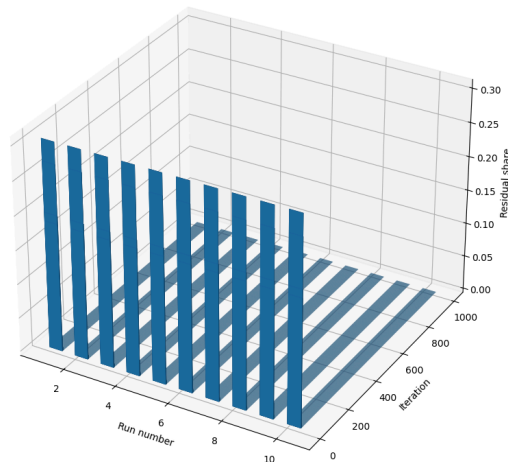
1000×1000 matrices of floating-point values from interval $[0, 1)$ (matrices returned by `numpy.random.rand(1000, 1000)`)^[15]



1000×1000 matrices of floating-point values from interval $(-100, 100)$ (matrices returned by `numpy.random.rand(1000, 1000) + numpy.random.randint(-99, 99, size=(1000, 1000))`)^[16]



1000×1000 matrices of integer values from interval $[-100, 100]$ (matrices returned by `numpy.random.randint(-100, 100, size=(1000, 1000)).astype(np.float64)`) ^[17]



1000×1000 Hilbert matrices (matrices returned by `scipy.linalg.hilbert(1000)`) ^[18]

4. Discussion

Though the particularity of applications ought not to be associated with rigorous and accurate research, a bias towards certain suggestions can be demonstrated. For example, with optimistic benchmarking the implementation against the selected counterparts on some types of matrices more pessimistic benchmark numbers for Hilbert matrices coexist - without known context, this is non-obvious; the colloquial concept of accuracy as overall characteristic of tool has not been considered.

External sources

-
1. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L122-L148> ↩
 2. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L150> ↩
 3. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L152-L153> ↩
 4. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L155-L158> ↩
 5. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L160-L162> ↩
 6. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L50-L54> ↩
 7. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L71-L75> ↩
 8. <https://github.com/doodosina-c/ldu-decomposition/src/main.pyx#L12-L26> ↩
 9. https://github.com/doodosina-c/ldu-decomposition/benchmark_reader#L47-L52 ↩
 10. https://github.com/doodosina-c/ldu-decomposition/data/comparison_with_other_tools/float.bin ↩
 11. https://github.com/doodosina-c/ldu-decomposition/data/comparison_with_other_tools/float-int.bin ↩
 12. https://github.com/doodosina-c/ldu-decomposition/data/comparison_with_other_tools/int.bin ↩
 13. https://github.com/doodosina-c/ldu-decomposition/data/comparison_with_other_tools/hil.bin ↩
 14. https://github.com/doodosina-c/ldu-decomposition/log_reader#L33-L35 ↩
 15. <https://github.com/doodosina-c/ldu-decomposition/data/statistics/float.bin> ↩
 16. <https://github.com/doodosina-c/ldu-decomposition/data/statistics/float-int.bin> ↩
 17. <https://github.com/doodosina-c/ldu-decomposition/data/statistics/int.bin> ↩
 18. <https://github.com/doodosina-c/ldu-decomposition/data/statistics/hil.bin> ↩